Signo: Real-Time Isolated Sign Language Recognition

Temporal Convolution and Mixup in a Landmark-Based Browser Recogniser (Version 4)

5th Semester B.Tech. Computer Science & Engineering | Section A
Machine Learning Laboratory (CSE 3152) | Manipal Institute of Technology, Manipal

Submitted By: John Soni Thomas (CSE-A, Roll No. 40, Reg. No. 240905338)
Mokshagna Y (CSE-A, Roll No. 42, Reg. No. 240905354)

Abstract

Signo is a browser application that recognises isolated American Sign Language (ASL) words from a webcam, using MediaPipe Holistic landmarks and a neural network. Our previous 1,000-sign model, version 2, averaged two Bidirectional LSTMs. In this project, we diagnosed its errors and ran 26 recorded experiments, with validation data alone deciding every choice. The diagnosis showed that the main limit is data per sign, so we tested methods that make a model generalise from few examples. Mixup, a temporal convolution network (TCN) and adding the validation clips to the final training gave real gains. Face landmarks, more frames, a frame embedding and motion input did not. The final model combines one TCN and one BiLSTM and keeps the same 30 x 100 input. To measure it fairly, we downloaded 8,134 official test clips that had never been used, and scored them once after every choice was final. It reached **83.2% top-1 and 96.4% top-5** on signers it never saw, against 74.1% and 92.6% for version 2 on the same clips, a gain of 9.1 points. It is also faster in the browser, at 68 ms per prediction against 116 ms. Accuracy on a real webcam has not yet been measured.

1. Introduction

Word-level sign recognition is difficult for two reasons. Signs are defined by movement over time, and public datasets offer few videos per word, so models can easily memorise the training examples. A score is only meaningful when the model is tested on people it has never seen, because that is what happens when a new person stands in front of the camera.

Version 2 of Signo scored 73.4% top-1 on 1,000 signs, and earlier tests of convolution-based models had suggested they could not compete with the BiLSTM. In this project, we questioned both results. We rebuilt the experiment process so that each idea was tested fairly, measured uncertainty for each comparison, and kept the test clips out of every decision.

Signo performs **isolated sign recognition, not continuous translation**. It recognises one word at a time from a fixed list. It does not follow natural signing or produce grammatical sentences.

2. Project Objectives

- **Objective 1:** Find out why the existing 1,000-sign model makes mistakes, by auditing its data, labels, features and validation errors.
- **Objective 2:** Improve top-1 accuracy and macro F1 through controlled experiments on architecture, augmentation and features, informed by published work and decided on validation data only.
- **Objective 3:** Measure the final model once on unseen test clips, and deploy it in the browser only after it matches Python on identical inputs.

3. Dataset Description & Anti-Leakage Data Engineering

We used ASL Citizen [1], which contains 83,399 videos of 2,731 signs from 52 signers, with an official split in which no signer appears in more than one set. We kept the same 1,000 signs as version 2. The licence does not allow its videos to be redistributed or substantially reproduced, so no dataset video frames appear in this report.

Check	Result
Train / validation clips	14,626 / 3,763, from 35 / 5 signers
Test clips	3,983 used in earlier versions, and 8,134 never used before, from the same 11 signers
Signers shared between any two splits	0

Check	Result
Duplicate video IDs, clips with identical features, non-finite values	0, 0, 0
Training clips per sign	11 to 17, a ratio of only 1.5, so class weighting has little to correct
Label order	Equal to the sorted training labels and to the browser's labels. Checked by an assertion in every run
Feature pipeline	Features rebuilt from raw landmarks match the stored values on 400 of 400 clips, largest difference 0.0007

Table 1: Audit of the 1,000-sign dataset.

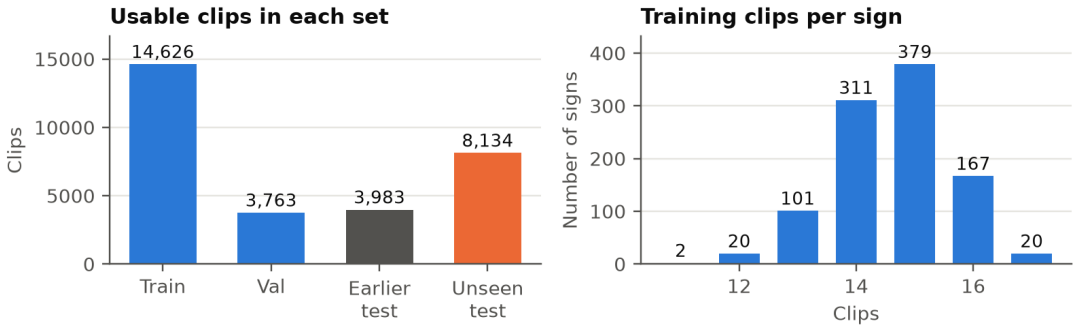


Figure 1: Left, usable clips in each set. Right, training clips per sign.

What the Diagnosis Found

- **Too little data per sign.** The official training split holds no more clips for these signs, and we already had all of them. On 5,000 training clips, a BiLSTM reaches 55.5%. On all 14,626, it reaches 78.0% with longer training.
- **Memorising.** Version 2's members score 97.5% and 95.4% on clean training clips against 79.4% on validation for the pair, so the real gap is about 18 points. **This corrects our version 2 report**, which quoted about 6 points by comparing augmented training clips with clean validation clips.
- **Lost hand tracking.** Version 2's top-1 falls from 84.5% when under 10% of a clip has no visible hand to 62.4% when over 30% does. This is a correlation, so we tested it directly.
- **Camera shape.** The features did not correct for video shape, and one test signer's clips are all 16:9, against almost none in training. Simulating 16:9 on validation dropped top-1 from 79.38% to 76.53%.
- **Earlier convolution tests were unfair.** They stopped at 34 to 72 epochs on a time budget, while the BiLSTM was still improving at 180.

Protecting the Test Set

The 3,983 test clips from earlier versions had already been scored twice, so they are not untouched. The official split holds more test clips for these signs, because version 1 capped its sample at 4 clips per sign to save storage. We downloaded the other 8,161 clips, of which 8,134 were usable, and did not score them until every choice was final. The evaluation script logs its run and refuses to run again. Test clips are also removed from the data before any training process starts.

4. Methodology & Model Architectures

The system has two connected parts, shown in Figure 2. MediaPipe [8] finds the body and hand points. Each frame gives the same 100 numbers as earlier versions: 7 upper-body points, 21 points per hand (each with x and y) and 2 hand-present flags, centred on the shoulders and scaled by shoulder width. Each clip is trimmed and resampled to 30 frames.

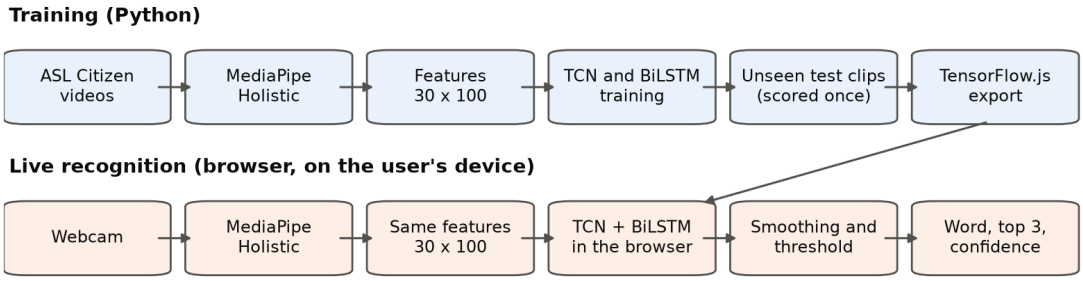


Figure 2: The training pipeline and the live browser pipeline.

Branch A: Automated Experiments and Mixup

An unattended controller, `auto_optimize.py`, trained runs from a queue, scored each on validation and compared it with a reference, using a paired bootstrap on shared clips to give a 95% interval. It also queued follow-ups from rules, for example trying more epochs when the best epoch fell near the end. Quick tests on 5,000 clips came first, and promising ideas then ran on all clips.

Mixup [2] blends two training clips and their labels in a random proportion, so the model cannot simply memorise individual clips. We added it to the existing changes applied to training clips: mirroring, time warping, small changes to size, rotation and position, and briefly blanking a hand. Mixup makes the model under-confident, with mean confidence of 74% against 82% accuracy, which would make the app show fewer words. We corrected this with temperature scaling [3], fitted on validation and built into each member's last layer, so the browser code did not need to change.

Branch B: Temporal Convolution Network and BiLSTM

A TCN [4] uses convolution filters along time, with gaps between filter positions (dilation of 1, 2, 4, 1, 2, 4) that let later layers see longer stretches of movement. A BiLSTM [7] instead reads the frames in both directions with a running memory. Our first TCN stayed at chance level. Testing without augmentation showed a setup fault, so we normalised before each block and used a wider final layer with less dropout. The two members of the final model are:

```
TCN: Input (30, 100) → Dense (128) → 6 residual Conv1D blocks (128 filters, width 5)
          → Average + Max pooling → Dense (256, ReLU) → Softmax (1,000)

BiLSTM: Input (30, 100) + frame-to-frame change → Bidirectional LSTM (128)
                    → Dense (128, ReLU) → Softmax (1,000)
```

Their outputs are averaged in one TensorFlow.js model of 1,851,136 parameters. We selected the combination by a rule fixed in advance: add the member that most improves validation top-1, and stop when a member adds under 0.3 points. Also fixed in advance, the chosen members were retrained with the same settings on training plus validation clips (18,389 clips from 40 signers), because the diagnosis pointed to data as the main limit. That is the deployed model. The training-only version is kept for comparison.

5. Experimental Results & Performance Comparison

We trained on a laptop CPU. Table 2 lists the quick tests, each compared with the same control. An interval that excludes zero indicates a real difference on these clips.

Change tested (5,000 clips)	Val Top-1	Macro F1	Against control, points (95% interval)	Decision
Control: BiLSTM, 128 units	**55.5%**	**53.5%**		
Mixup + body variation	60.9%	58.5%	+5.4 (+4.0 to +6.8)	Test on all clips
Mixup	60.0%	57.6%	+4.4 (+3.1 to +5.8)	Keep
Label smoothing	57.5%	55.1%	+2.0 (+0.6 to +3.4)	Test on all clips
Motion input	56.8%	54.6%	+1.2 (-0.1 to +2.6)	Test on all clips
Bridging hand gaps	56.2%	53.9%	+0.7 (-0.6 to +2.1)	Reject

Change tested (5,000 clips)	Val Top-1	Macro F1	Against control, points (95% interval)	Decision
Hand size and arm length variation	55.8%	53.6%	+0.3 (-1.1 to +1.6)	Reject
Blended frame sampling	55.6%	53.4%	+0.1 (-1.2 to +1.4)	Reject
45 frames instead of 30	52.9%	50.5%	-2.7 (-4.1 to -1.2)	Reject
Face points	48.7%	46.2%	-6.8 (-8.2 to -5.3)	Reject
Face points + bridging	46.1%	43.6%	-9.4 (-10.9 to -7.9)	Reject
Frame embedding	43.4%	41.0%	-12.1 (-13.7 to -10.6)	Reject

Table 2: Quick tests on 5,000 training clips, on all 3,763 validation clips.

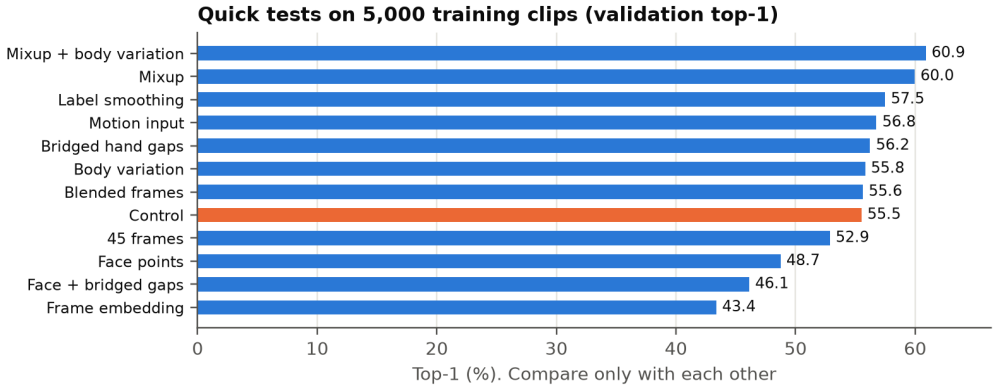


Figure 3: The quick tests. The two convolution tests are left out, because they failed from a setup fault.

Model (all 14,626 training clips)	Epochs	Val Top-1	Val Top-5	Macro F1	Gap (points)	Parameters
TCN, mixup 50% (selected)	180	84.3%	96.6%	83.9%	15.7	1,322,344
TCN, mixup 100%, seed 1	130	83.8%	96.9%	83.4%	15.8	1,322,344
TCN, mixup 100%, seed 2	130	83.6%	96.9%	83.1%	16.2	1,322,344
BiLSTM, mixup 100%	240	83.2%	96.9%	82.5%	13.1	396,392
BiLSTM, mixup 100%	180	82.1%	96.5%	81.4%	12.4	396,392
BiLSTM, mixup 100%, seed 1	180	82.0%	96.8%	81.5%	12.6	396,392
BiLSTM 192 units, mixup 50%	180	81.3%	96.3%	80.6%	17.8	716,968
BiLSTM, mixup 100%, motion (selected)	180	81.3%	96.4%	80.6%	13.1	518,792
BiLSTM, mixup 100%, smoothing	180	81.2%	96.5%	80.6%	11.6	396,392
BiLSTM, mixup 50%, smoothing	180	80.8%	96.0%	80.4%	14.1	396,392
BiLSTM, mixup 50%	180	80.5%	95.3%	79.9%	15.9	396,392
BiLSTM, mixup 50%, body variation	180	79.7%	95.5%	79.2%	15.2	396,392
Earlier single BiLSTM (reference)	180	78.0%	94.5%	77.5%	19.5	396,392

Table 3: Runs on all training clips. Gap is clean training accuracy minus validation accuracy. The 180-epoch TCN run stopped at 158 under a time cap. Stopped early and not listed: the two-layer BiLSTM (epoch 28), half-strength 300 epochs (epoch 175), half-strength motion (epoch 6), convolution with attention (epoch 87, 68.7%) and a 96-channel TCN (epoch 17).

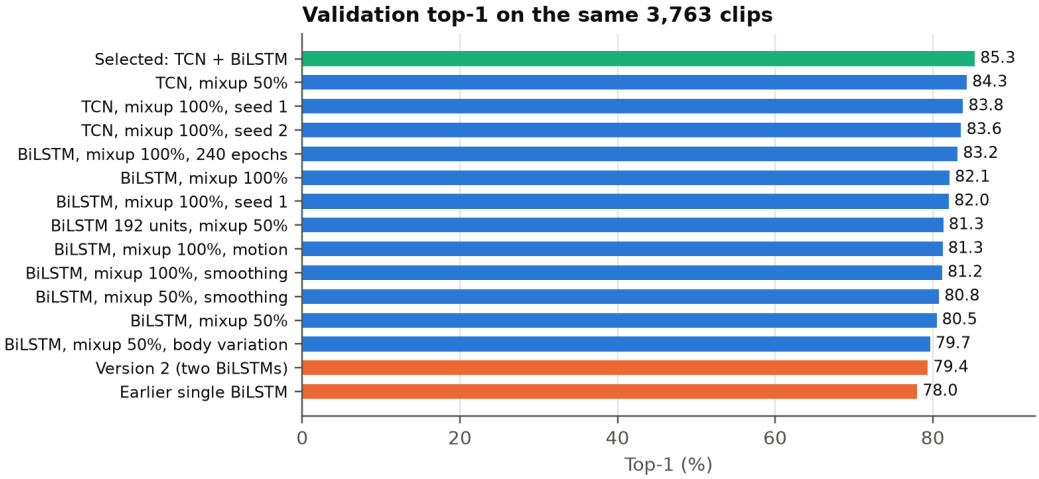


Figure 4: Validation top-1 of all full runs and the selected combination, on the same clips.

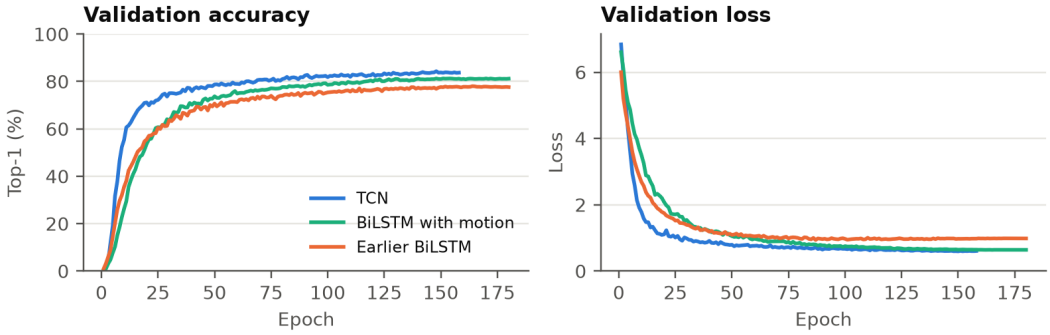


Figure 5: Validation accuracy and loss during training for both selected members and the earlier BiLSTM.

Combining the TCN and the BiLSTM with motion input gave 85.33% on validation, with macro F1 of 84.87%. Adding a third member added only 0.29 points, under the 0.3-point rule, so the model stayed at two. Table 4 gives the final results.

Test set	Model	Top-1 (95% interval)	Top-5	Macro Precision	Macro Recall	Macro F1	Weighted F1
Unseen (8,134 clips)	**Version 4 (Proposed)**	**83.2% (82.4% to 84.0%)**	**96.4%**	**84.4%**	**83.2%**	**82.8%**	**82.9%**
Unseen (8,134 clips)	Version 4, training clips only	81.1% (80.2% to 82.0%)	95.6%	82.2%	81.0%	80.6%	80.8%
Unseen (8,134 clips)	Version 2	74.1% (73.2% to 75.1%)	92.6%	75.4%	74.2%	73.6%	73.7%
Earlier (3,983 clips)	**Version 4 (Proposed)**	**83.8% (82.7% to 84.9%)**	**96.6%**	**85.2%**	**83.8%**	**83.1%**	**83.1%**
Earlier (3,983 clips)	Version 4, training clips only	81.8% (80.5% to 83.0%)	95.6%	84.1%	81.8%	81.3%	81.3%
Earlier (3,983 clips)	Version 2	73.4% (72.0% to 74.7%)	92.0%	75.4%	73.4%	72.5%	72.5%

Table 4: Final performance. Random guessing scores 0.1% top-1. The unseen clips give the unbiased score, and were scored once.

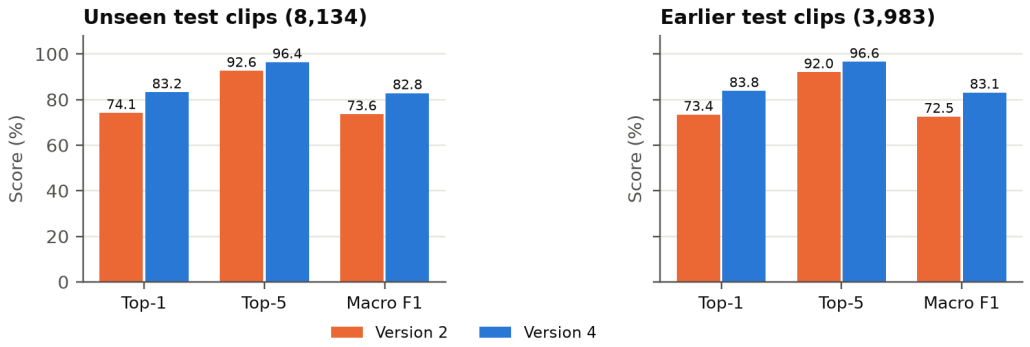


Figure 6: Version 2 and version 4 on both test sets.

Key findings from the experiments:

1. **A real gain on unseen clips.** On the same 8,134 clips, version 4 is 9.1 points better than version 2 (95% interval 8.2 to 9.9). On the earlier clips, the gain is 10.4 points. Signs with recall of 50% or less fell from 171 to 88.

2. **The TCN gave the largest gain.** It scored 84.32% on validation, +6.3 (+5.1 to +7.5) points against the earlier BiLSTM. It also beat the best BiLSTM here, by +1.2 (+0.1 to +2.3) points, which is narrow. The earlier conclusion against convolution came from unfair tests.

3. **Mixup was next.** Blending every batch added +4.1 (+3.0 to +5.3) points to the same BiLSTM, and a second seed gave 82.04% against 82.12%, so the gain is stable.

4. **More training data helped.** Adding the validation clips raised top-1 on the unseen clips from 81.1% to 83.2%, a gain of 2.1 points (95% interval 1.5 to 2.7), which supports data being the main limit.

5. **Some published ideas did not transfer.** Face points cost 6.8 points, although top Kaggle solutions use them [5]. One possible cause is that face shape helps a model recognise the signer, since the training gap widened, but we did not test this. Frame embedding also lost here, against a published gain [6], though only on 5,000 clips with a BiLSTM. Motion input did not help by itself. The rule chose that BiLSTM because it combines well with the TCN, not because motion helps.

6. **The model still memorises.** The TCN scores 99.97% on clean training clips against 84.32% on validation, a gap of 15.7 points. The gap has narrowed only somewhat from about 18.

7. **Every test signer improved.** As Figure 7 shows, the lowest signer rose from 66.7% to 76.6%, but signers still range up to 92.0%. The signer filmed in 16:9 rose from 78.6% to 87.6%. Correcting for video shape probably contributes, but we did not separate it from the model change.

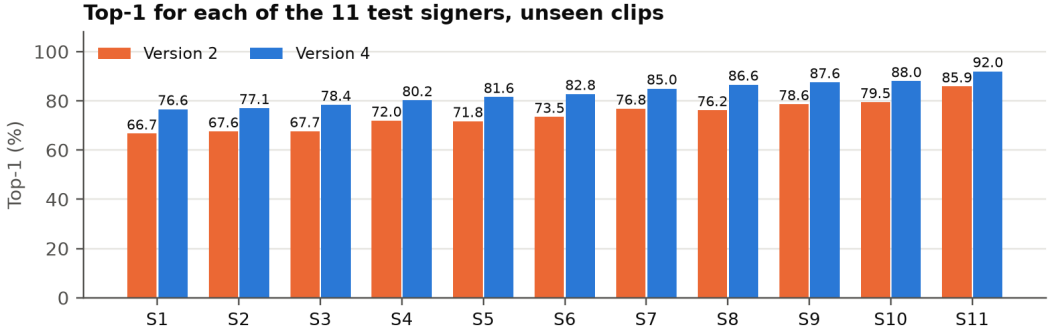


Figure 7: Top-1 for each test signer on the unseen clips, sorted by version 4 score. Signers are numbered for this chart.

8. **Validation was optimistic.** The training-only model scored 85.3% on validation and 81.1% on the unseen clips. Part of this 4.2-point drop is likely from many comparisons on the same validation clips, and part from different signers.

9. **Similar signs remain the next limit.** Top-5 is 13.2 points above top-1, and the top 100 confused pairs hold 34% of errors, including *earth* and *planet*, *assist* and *assistant*, *experience* and *experienced*, and *register* and *rule*. Some pairs may be the same or nearly the same sign. Each sign has only about 8 unseen test clips, so per-sign scores are rough.

Best 10 signs	F1	Worst 10 signs	F1
above	100.0%	equal	0.0%
important	100.0%	cost	13.3%
impact	100.0%	assistant	16.7%
idea	100.0%	even	16.7%
ice	100.0%	price	20.0%
hundred	100.0%	earth	25.0%
high	100.0%	planet	25.0%
improve	100.0%	way	25.0%
health	100.0%	few	28.6%
hate	100.0%	rescue	28.6%

Table 5: Best and worst signs on the unseen clips. 178 signs tied at 100% F1, so the best list is only a sample.

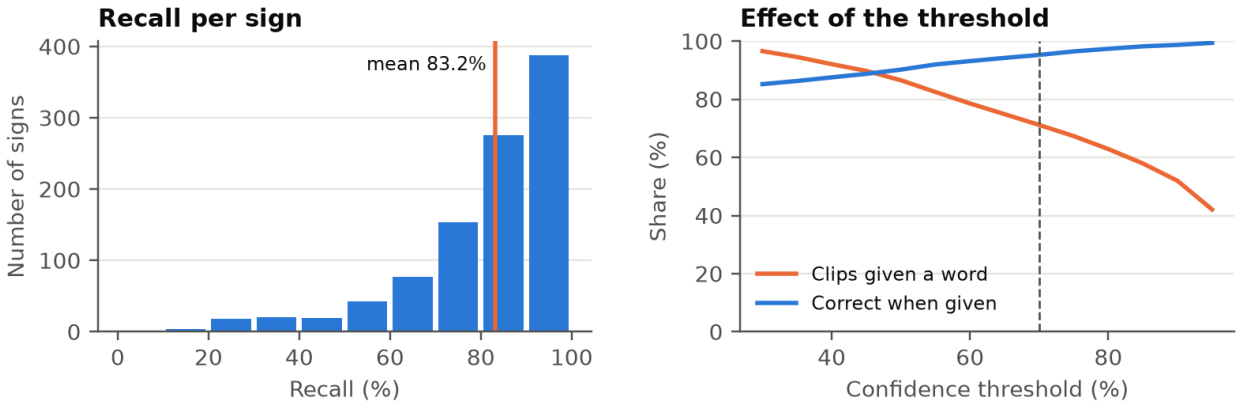


Figure 8: Left, the number of signs at each recall on the unseen clips. Right, the effect of the confidence threshold on the same clips. The dashed line marks the app's default.

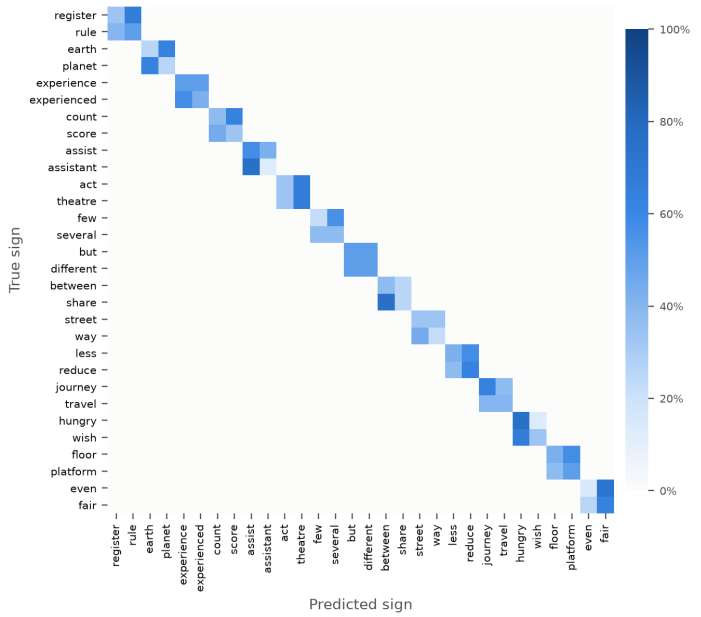


Figure 9: Test confusion matrix for 30 signs taken from the most frequent confusions. Errors involving other signs are not shown. The full matrix is in `results/final_test/final_fresh_confusion.csv`.

6. Browser Integration & Real-Time Verification

The model was exported to TensorFlow.js and added to the app as the default, with version 2 still selectable. Export exposed two problems, which we fixed: TensorFlow.js could not read an array as a layer's starting value, and it computed the wrong output length for causal padding. An equivalent layer gives identical numbers, and the validation score did not change. A word appears only when confidence passes the threshold (70% by default) and it stays on top for 2 consecutive checks. On the unseen clips, the 70% threshold gives a word for 71.2% of clips, and 95.3% of those words are correct.

Check	Result	Status
TF.js vs Python on identical sequences	Same top-1 and top-3 order on 300 of 300 test sequences. Largest probability difference under 0.000001	Passed
Test videos through the browser's whole pipeline	40 clips: 82.5% top-1, 92.5% top-5. Python 85% on the same clips. Same prediction on 95% of clips	Passed
Prediction speed (idle machine)	68 ms mean, 72 ms p95. Version 2: 116 ms mean, 119 ms p95	Passed
Live app loop, test videos as a simulated camera	8 clips, first pass: 6 right, 1 wrong (book read as school), 1 with no word, and 1 extra wrong word. Median 217 ms between predictions and 17.4 camera frames per second in this run, against 4 to 11 for version 2. Six signs were picked for good earlier results, so this is not an accuracy score	Passed as a functional check
Model switching and fallback	Labels and output size stay matched. If the new model is missing or corrupt, the app opens with the previous one	Passed
Automated tests	All 23 existing tests pass	Passed
Camera-shape correction	Formula checked in Python to within 0.0000004. 4:3 video is unchanged. Not yet run on a real widescreen camera	Partly checked
Real webcam accuracy	Version 4 not yet measured	Not measured

Table 6: Verification of the browser application.

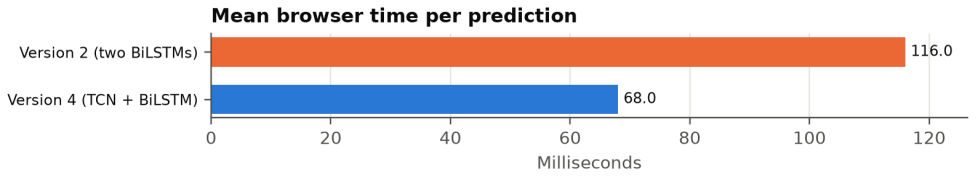


Figure 10: Mean prediction time in Chrome with WebGL.

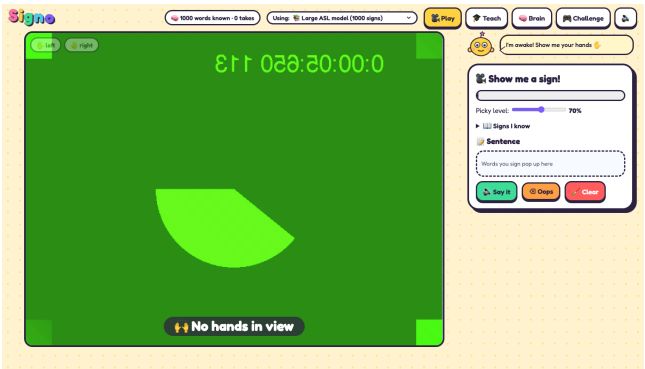


Figure 11: The Signo interface. The camera feed here is Chrome's built-in test pattern, so the app correctly reports that no hands are in view.

7. Practical Deployment: Choosing a Model

Signo runs entirely on the user's device, so no video is uploaded. An internet connection is needed only to load TensorFlow.js and MediaPipe. The model files total 7.2 MB. The main models serve different purposes:

- **Large ASL model (1,000 signs, default):** version 4, with 83.2% top-1 and 96.4% top-5 on unseen signers.
- **Accurate ASL model (50 signs):** unchanged from version 2, with 91.2% top-1 and 97.6% top-5 on its own test set. It

is an easier task and a different test set, so the two are not directly comparable.
- **Previous large ASL model:** version 2, kept for comparison.

Known Limitations

- **Live accuracy is unknown:** all test scores come from fluent signers in dataset videos. A beginner's signing at a webcam may score lower.
- **Same test signers:** both test sets come from the same 11 signers, so the score may not hold for very different signers or cameras.
- **Earlier test clips are not untouched:** the 3,983 clips have now been scored in three versions. Quote the unseen clips' score.
- **Single seeds in the final model:** each member comes from one seed, though other seeds gave similar results. The final retrained model has no validation score of its own.
- **Limited search:** many runs used one seed, and we did not test pretraining on the other 1,731 signs or give graph networks a fair test.
- **No facial expression:** the nose is the only face point used, although facial expressions carry real meaning in ASL. Adding face points lowered accuracy in our test.
- **Fixed, unfiltered vocabulary:** signs outside the list are forced towards the closest known sign. The list was chosen by word frequency and includes some offensive words.

8. Conclusion & Future Work

In this project, we improved Signo's 1,000-sign model by first finding its main limit, then testing each idea with measured uncertainty. A temporal convolution network and mixup gave most of the gain, and training on the validation clips added more, all while keeping the same 30 x 100 input. On 8,134 test clips never used before, top-1 rose from 74.1% to 83.2% and macro F1 from 73.6% to 82.8%, and predictions became faster. Just as usefully, several appealing ideas were measured and rejected, and one earlier conclusion against convolution models was overturned.

For future work, we plan to measure webcam accuracy with real users, test the camera-shape correction on a widescreen camera, try pretraining on the other 1,731 ASL Citizen signs, review near-duplicate sign labels, and add a "not signing" class.

References

- **[1]** A. Desai et al., 'ASL Citizen: A Community-Sourced Dataset for Advancing Isolated Sign Language Recognition', Advances in Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks Track, 2023.
- **[2]** H. Zhang, M. Cisse, Y. N. Dauphin, and D. Lopez-Paz, 'mixup: Beyond Empirical Risk Minimization', International Conference on Learning Representations (ICLR), 2018.
- **[3]** C. Guo, G. Pleiss, Y. Sun, and K. Q. Weinberger, 'On Calibration of Modern Neural Networks', International Conference on Machine Learning (ICML), 2017.
- **[4]** S. Bai, J. Z. Kolter, and V. Koltun, 'An Empirical Evaluation of Generic Convolutional and Recurrent Networks for Sequence Modeling', arXiv:1803.01271, 2018.
- **[5]** H. Sohn, 'Google Isolated Sign Language Recognition, 1st Place Solution', Kaggle, 2023. github.com/hoyso48/Google---Isolated-Sign-Language-Recognition-1st-place-solution
- **[6]** T. Vandendriessche, M. De Coster, A. Lejon, and J. Dambre, 'Representing Signs as Signs: One-Shot ISLR to Facilitate Functional Sign Language Technologies', arXiv:2502.20171, 2025.
- **[7]** S. Hochreiter and J. Schmidhuber, 'Long Short-Term Memory', Neural Computation, vol. 9, no. 8, 1997.
- **[8]** C. Lugaresi et al., 'MediaPipe: A Framework for Building Perception Pipelines', arXiv:1906.08172, 2019.